Week 1 — Team Formation, Project Planning & Development Setup

Objective

The objective of Week 1 is to establish the project team, understand the assigned microcontroller architecture, finalize the development technology, divide responsibilities, and establish a professional GitHub-based development workflow.

No major coding or simulator implementation is expected in Week 1.

---

1. Form Your Project Team

- Each team must have 4 members.

The team must identify:

- **Team Leader**
- Team members
- Primary responsibility of each member
- Secondary/support responsibility of each member
- The division of work must be documented. Every member must have a clearly identifiable contribution to the project.

---

2. Create a Public GitHub Repository

- The Team Leader must create one Public GitHub repository for the project.

The following are mandatory:

- Repository must be **Public**.
- Team Leader must be clearly identified.
- All 4 team members must be added as collaborators.
- Every student must use their **own GitHub account**.
- Every student must make meaningful contributions through Git.
- Do not use one student's account for the entire project.

Minimum repository structure

```
None
project-root/
├── README.md
├── .gitignore
├── docs/
│   ├── meetings/
│   ├── decisions/
│   └── weekly-status/
├── src/
├── tests/
└── programs/
```

The structure may be modified later as the project develops.

---

3. Create the README

The initial `README.md` must clearly contain:

- Project title
- Problem Objective
- Problem Statement
- Project scope
- Microcontroller being simulated
- Team members
- Team responsibilities
- Selected programming language
- Initial system architecture
- Initial development plan

The README must be written by the team and understood by all members.

---

4. Add `.gitignore`

An appropriate `.gitignore` file must be added to the repository.

It should prevent unnecessary files such as:

- IDE-generated files
- Temporary files
- Build files
- Cache files
- Generated files
- Operating-system-specific files

from being committed to the repository.

---

5. Select the Programming Language

- The programming language must be finalized during Week 1.

Preferred language: Java

- Java is the preferred language for this project.
- If the team chooses another language, the team must document the reasons, advantages, and limitations.
- Selected language
- Reason for selection
- Advantages for this project
- Limitations
- Why Java was not selected
- Effect on project development

The alternative language requires **faculty approval**.

Once finalized, the programming language should not be changed later without documented justification and faculty approval.

---

6. Study the Assigned Microcontroller

- Before implementation begins, the team must understand the basic architecture of its assigned microcontroller.

The Week-1 study should cover:

- CPU/register organization
- Program Counter
- Stack Pointer
- Status/flag information
- Memory organization

- Stack mechanism
- Instruction categories
- GPIO
- Timer
- Interrupt mechanism

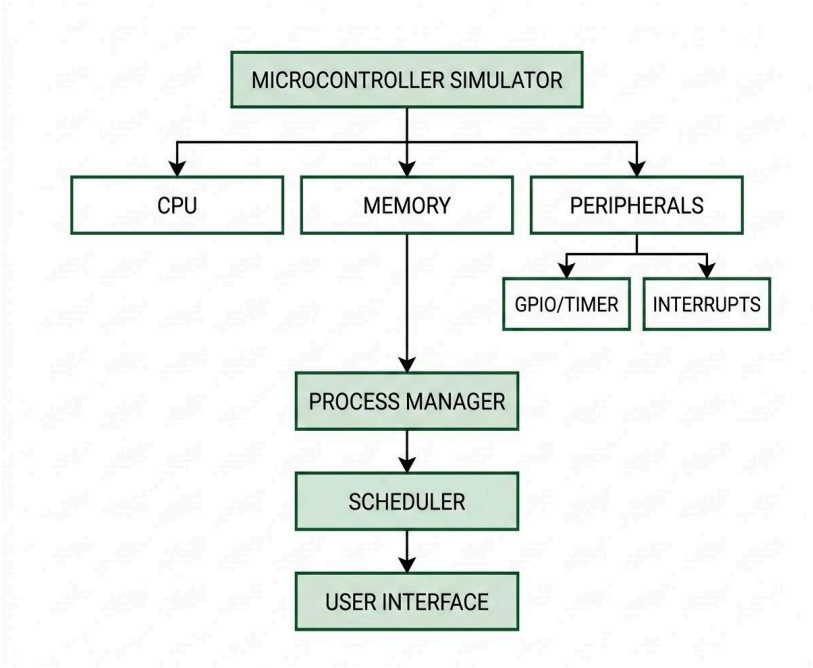
Students are **not expected to study or implement the complete processor**.

The purpose is to understand the features required for the simulator.

---

7. Design the Initial Simulator Architecture

- Prepare an initial block diagram showing how the major software components will interact.



- The diagram is an initial design and may be improved in later weeks.

---

8. Divide the Work

- The team must clearly assign responsibilities and commit them to GitHub.

A suggested division is:

Member	Primary Responsibility	Supporting Responsibility
Student 1 — Team Leader	CPU & Instruction Execution	Integration & GitHub
Student 2	Memory & Stack	CPU support
Student 3	Data Structures & Process Management	Testing
Student 4	OS Scheduling & Context Switching	UI & Integration

The team may modify this allocation based on its strengths, but **all five members must have meaningful technical responsibilities**.

The responsibility allocation must be committed to GitHub.

---

9. Individual Contribution is Mandatory

- Every student must contribute at least one meaningful GitHub commit by the end of Week 1.

By the end of Week 1:

> **Every student must have at least one meaningful GitHub commit.**

Examples of acceptable contributions include:

- Processor architecture study
- Register/memory documentation
- Instruction-set analysis
- Data-structure design

- Process-management design
- Scheduler planning
- UI design
- Test-plan preparation
- Project documentation

Creating an empty file, changing formatting, or making insignificant edits will **not** be considered a meaningful contribution.

Every student should be able to explain their contribution during the Week-1 review.

---

10. Project Communication Rules

- The project must maintain a professional and traceable communication process.

GitHub Issues

Use GitHub Issues for:

- Technical questions
- Problems
- Bugs
- Design concerns
- Tasks
- Dependencies
- Important decisions
- Features requiring discussion

Google Classroom / Shared Google Documents

Use the documents provided through Google Classroom for:

- Meeting minutes
- Brainstorming
- Detailed discussions
- Weekly status reports
- Formal project documentation

WhatsApp

WhatsApp may be used for informal coordination if necessary.

However:

> **Technical discussions and critical project decisions must not be maintained only in WhatsApp.**

If an important decision is discussed informally, the final discussion and decision must be transferred to the official project record.

"We discussed it in WhatsApp" is not considered official project documentation.

---

11. Maintain Meeting Minutes

- Every important team meeting must have documented minutes recording date, participants, agenda, discussions, and decisions.

The minutes should include:

Date:
Meeting Number:
Participants:

Agenda:
1.
2.

Discussion:
...

Decisions:
1.
2.

Action Items:
Student 1:
Student 2:
Student 3:
Student 4:

Next Meeting:

Important technical decisions must be recorded in the project documentation.

---

12. Create and Use GitHub Issues

During Week 1, the team should identify questions, concerns, or decisions that require discussion.

For example:

Issue: Instruction Representation

Problem:
How should instructions be represented internally?

Options:
1. Class-based representation
2. Table-based representation

Discussion:
...

Decision:
...

Responsible:
Student 3

Status:
Open / Closed

Issues should have appropriate:

- Title
- Description
- Assignee
- Discussion
- Status

---

13. Weekly Git Branch

- Create a "week-01" branch for development and merge it into "main" after review.

Create the Week-1 branch:

week-01

All Week-1 work should be developed and committed to this branch.

At the end of the week:

week-01
↓
Review
↓
Testing
↓
Documentation
↓
Merge
↓
main

The accepted Week-1 work must be merged into `main`.

---

14. Document What Was Accepted

Before merging `week-01` into `main`, the team must document:

- Work completed
- Work tested
- Work accepted
- Work not completed
- Open issues
- Concerns
- Work carried forward to Week 2

For example:

Week 1 Acceptance

Completed:
- Repository setup

- Architecture study
- Language selection
- Initial design

Accepted:
- Initial simulator architecture
- Team responsibility allocation

Pending:
- Detailed CPU design
- Instruction implementation

Issues:
- #01 Language decision
- #02 Memory model

Carry Forward:
- Week 2 CPU implementation

---

15. Weekly Status Update

The team must submit a Week-1 status update.

It should contain:

- Planned work
- Completed work
- Pending work
- Issues encountered
- Decisions made
- Individual contributions
- Work accepted
- Work carried forward
- Week-2 plan

---

16. Week-1 Deliverables

By the end of Week 1, the following must be available:

GitHub

- Public repository
- Team Leader identified
- All team members added
- `README.md`
- `.gitignore`
- Initial folder structure
- `week-01` branch
- Meaningful commits from all members
- Week-1 branch merged into `main`

Project Planning

- Team responsibilities documented
- Microcontroller architecture studied
- Programming language finalized
- Initial simulator architecture diagram
- Initial development plan

Documentation

- Meeting minutes
- Important decisions
- Relevant GitHub Issues
- Week-1 status report
- Accepted work documented
- Pending work documented